

Codex 进阶学习指南

看懂配置 协议 权限和证据的边界

核心结论 schema 通过，只证明数据形状符合这份契约

学习目标

围绕十二个递进问题，把 Codex 的客户端、运行时、配置、认证、协议和模型访问拆开理解。已有的 MCP、状态外置和沙箱基础只做关联，把篇幅留给新的接口契约。

本次实际做过

2026-10-05 使用固定提交的两份官方 Draft-07 schema，校验合成 TOML 与请求对象。本次公开整理未重跑；不依赖真实账户、服务端或模型调用。

证明边界

原视频未观看、未核验。本报告是官方资料限定范围学习；没有启动 Codex CLI 或 App Server，没有创建线程、轮次，没有读取真实配置或凭证，没有验证模型资格、运行行为或费用。

从浅到深的十二个问题

- 01

进阶到底要看懂哪几层
- 02

订阅登录和 API 是一回事吗
- 03

Cloud 和远程客户端差在哪里
- 04

为什么能登录还可能不能调用
- 05

配置为什么写了却没有生效
- 06

Provider 字段为什么也会误导
- 07

沙箱和审批到底管什么
- 08

Thread Turn Item 怎么串起来
- 09

请求格式对就能直接发送吗
- 10

model list 能证明模型权限吗
- 11

JSON Schema 到底证明什么
- 12

怎样留下别人能复验的证据

先分清产品和用量路线

01 进阶到底要看懂哪几层

先辨认执行位置和协议层，再讨论模型

Codex 不只是一个模型名。客户端是你操作的界面；运行时负责会话、工具和执行；模型完成推理。Cloud 是托管任务环境，App Server 是把 Codex 能力接入自有客户端的协议接口。它们处在不同层。

把问题拆成“谁发请求、在哪执行、谁来推理、怎样授权”。例如同一个模型名出现在配置和目录中，仍不能回答代码在哪里运行、账户能否访问。本次只验证两份公开 schema 的数据结构。



图解：客户端、执行运行时与模型是三个观察层；箭头不代表已经联网执行。

依据 Codex Cloud / App Server 协议

02 订阅登录和 API 是一回事吗

登录路线影响用量归属，不能推导无限或免费

认证是确认“你是谁”。官方提供 ChatGPT 登录和 API key 等认证路线；ChatGPT 登录使用订阅相关的 Codex 访问，API key 路线使用 API 的用量计费。具体可用性、限制及组织策略要单独核查。

OAuth 是授权客户端取得一定访问权限的机制，不是把订阅变成通用免费 API。官方面向开源或本地托管应用提供明确授权的 ChatGPT 登录集成，但有支持范围和安全要求。本实验没有登录、读取令牌或发起计费请求。



图解：认证路线可以分叉；任何分支都不能凭名称推断额度、价格或具体模型权限。

依据 登录方式与计费边界 / ChatGPT 登录集成

执行位置与访问条件

03 Cloud 和远程客户端差在哪里

远程连接是一条通道，Cloud 是一种执行安排

Cloud 任务由托管环境处理，官方文档说明使用 Cloud 需要 ChatGPT 登录。远程客户端则描述界面与 Codex 运行时之间的连接关系；它不自动把运行时变成 Cloud，也不自动提供订阅或额外权限。

可以把界面理解为遥控器，把运行环境理解为工作台。遥控器在别处，不会改变工作台的文件、网络或账户规则。判断数据会去哪里，应核对真正的运行主机、传输边界与认证方式，不能只看 UI。



图解：执行环境与连接方式是两条判断轴；本图只解释概念，没有建立远程连接。

依据 [Codex Cloud / App Server 协议](#) / [登录方式与计费边界](#)

04 为什么能登录还可能不能调用

身份、权限、模型资格和费用要分别举证

授权是决定“允许做什么”；模型资格是账户或组织能否使用某个模型；计费归属说明用量记到哪里。它们与认证相关，却不是同一个事实。一个成功登录记录不能同时证明另外三项。

排查时逐层问：身份是否有效？这项操作是否被允许？这个账户是否有目标模型资格？走的是哪条用量路线？本包的离线通过结果只回答字段形状，不替代以上任何一层，也不提供绕过限制的方法。



图解：四个检查点需要各自的证据，前一项成立不会自动使后一项成立。

依据 [登录方式与计费边界](#) / [高级配置](#) / [App Server 协议](#)

配置如何被解释

05 配置为什么写了却没有生效

先看来源、优先级与信任，再看字段拼写

配置叠加是将多个来源合成为最终设置。来源包括命令行覆盖、项目层、profile 文件、用户配置等；这里不枚举完整优先级。项目层还受目录层级与信任约束。一个 TOML 文件能解析，只证明文本语法成立。因此要分三次检查：TOML 能否解析；解析后的对象是否符合版本对应的 schema；真正加载时该来源是否被采用并允许覆盖。本实验只做前两项，不读取或写入真实配置，更不声称看到最终生效配置。



图解：本实验走到前两站；运行时加载和最终行为没有测试。

依据 [配置基础](#) / [高级配置](#) / [固定版 ConfigToml schema](#)

06 Provider 字段为什么也会误导

配置路由描述不等于真实服务能力

Provider 是模型服务的连接描述，可能包含服务地址、协议形式与认证相关配置。requires_openai_auth=false 也不表示无需认证。结构合规只说明字段被 schema 接受，不能证明端点可达、协议兼容、凭证有效或模型可用。

2026-10-05 核查的官方高级配置明确说明：model_provider 与 model_providers 属于用户级配置，项目级 .codex/config.toml 中这些键会被忽略。这个结论来自文档，不是本次运行时测试；不要用项目文件的 schema 通过反驳它。



图解：连接描述之后还有加载策略、网络协议与账户条件，schema 不负责证明这些。

依据 [高级配置](#) / [固定版 ConfigToml schema](#)

执行约束与协议对象

07 沙箱和审批到底管什么

沙箱限定能做的事，审批决定何时询问

沙箱是对执行环境的技术限制，例如文件写入和网络范围；审批策略规定哪些动作要等待同意。两者是独立维度，减少询问不会凭空扩大环境能力。组织策略或上层系统还可以施加更严格的约束。

旧配置示例也要看日期。2026-10-05 核查的官方文档把 untrusted 审批策略标为已退役；固定版 schema 中仍有相关枚举，并不使其成为当前推荐。我们的样例用于形状测试，没有修改任何沙箱、审批或全局设置。



图解：一个动作必须同时满足环境边界和审批条件；旧枚举可接受不等于当前可推荐。

依据 高级配置 / 固定版 ConfigToml schema

08 Thread Turn Item 怎么串起来

用对象层级理解会话，用事件理解进展

在 App Server 中，Thread 表示会话容器；Turn 是一次用户输入触发的处理轮次；Item 是该轮中的输入、输出或执行记录。JSON-RPC 是请求与响应使用的方法、参数和标识约定。

请求用 id 对应响应中的 result 或 error；通知没有 id。一次轮次可以产生多条 item 与通知，收到一条消息不代表全部完成。官方 App Server 的 JSON-RPC 2.0 消息省略 jsonrpc 头。本次只校验合成请求，没有发包或创建会话。



图解：层级说明数据归属；示意对象没有在任何真实账户或服务里创建。

依据 App Server 协议 / 固定版 ClientRequest schema

生命周期与模型目录

09 请求格式对就能直接发送吗

格式合规不代表生命周期合规

生命周期是协议规定的先后状态。官方基本流程为 initialize 请求及响应，再发 initialized 通知；随后 thread/start 建会话，turn/start 开始一轮，最后检查 turn/completed 的状态与结果。顺序与终态需要运行时证据。

我们故意使用不存在的合成标识，展示“能通过 schema”与“真实对象存在”互不等价。校验器是静态检查者，不维护服务端会话状态。测试成功的含义是反例符合预期，而不是后端执行成功。



图解：离线校验只能检查第一站，不能跳过初始化、对象存在性和动作结果。

依据 App Server 协议 / 固定版 ClientRequest schema

10 model list 能证明模型权限吗

目录里能看见，与账户能使用是两份证据

model/list 是 App Server 的模型目录查询方法。目录可以帮助客户端展示模型和选项，但目录条目不是一次授权判决，更不是成功推理或费用证明。本次连这个目录接口都没有调用。

一个合成模型名可能是合法字符串，因此通过 schema。要证明真实可用，需在明确授权的环境与账户里核对官方可用性，并用符合权限与预算的操作验证；本指南不把这种在线验证伪装成本次离线结果。



图解：名称、目录、资格和实调结果逐级增强；这里只验证第一项的结构条件。

依据 App Server 协议 / 登录方式与计费边界 / ChatGPT 登录集成

11 JSON Schema 到底证明什么

它验证契约的形状，不替运行时做决定

JSON Schema 是描述数据结构规则的标准；Draft-07 是这次两份官方 schema 声明的规范版本。校验器检查类型、必填键、枚举和分支等规则。TOML 先被解析为对象，再接受 ConfigToml schema 校验。

我们分别使用合法、非法与“形状合法但语义未知”的合成样例，观察通过和错误路径。真正的进阶点是划清证明边界：schema 的成功不是配置已采用、请求已执行、模型已授权或费用为零。



图解：校验器擅长结构规则；运行状态、账户决策与计费属于其他证据来源。

依据 [固定版 ConfigToml schema](#) / [固定版 ClientRequest schema](#)

12 怎样留下别人能复验的证据

固定来源，保留反例，让结论和证据逐项对齐

可复验是别人拿到相同材料，能重做检查并解释结果。本包固定官方 Git 提交，保留原始 schema、Apache 2.0 许可、字节哈希、合成样例和运行器。公开 schema 来源与当前动态文档分开标注。

安装清单里的版本号与某个源码提交不是自动等价的；安装清单只读过，CLI 没有执行。文档结论、源码契约与离线结果各有范围。下一步若做在线实验，必须另行授权、设定预算并重新留证。



图解：从来源到复验形成证据链；其中每一环都不能用“应该如此”替代实际记录。

依据 [固定版 ConfigToml schema](#) / [固定版 ClientRequest schema](#) / [上游 Apache 2.0 许可](#)

五项能力和五个具体用途

01 拆分产品与执行边界

先问客户端、运行时和模型各在哪一层，避免把“远程”和“Cloud”混成一种能力

02 核对身份和用量路线

分别记录认证、操作权限、模型资格和计费归属，不从订阅名称推断免费或无限

03 审查配置的实际含义

区分文本解析、schema 接受、来源生效和运行结果，发现旧示例与新文档的差异

04 阅读协议对象与状态

辨认 Thread、Turn、Item 与初始化流程，区分请求格式和服务端生命周期

05 设计可复验反例

把合法、非法和语义未知样例放进同一套离线验证，给每项结论标上证据边界

学完能拿来做什么

1. 审阅一份 Codex 配置教程：标出过期审批示例、错误配置层级与缺失的版本信息
2. 设计自有客户端的接入评审：先画握手与会话状态，再讨论 UI 和消息展示
3. 排查“名字写对却调用不了”：把字段错误、加载规则、账户资格与计费路线分开查
4. 给自动化流水线增加配置预检：在接触真实账户之前，先离线阻止明显类型与枚举错误
5. 整理知识库：把 schema 契约与既有证据状态、边界控制知识关联，避免重复新建概念

学习不是获得在线执行授权

这些用途从阅读、设计与离线检查开始。真实连接、凭证配置、模型请求和费用相关动作，仍需对应环境、权限和明确授权。

六步实验 从解包到正例

Bash 与 Python 3.11+。以下为 2026-10-05 新解包实测；本次公开整理未重跑。

01 解包并核对环境

目的 从独立 ZIP 开始，确认不是依赖原工作目录

```
python3 -m zipfile -e codex-advanced-exercise.zip replay
cd replay
python3 -I -c "import sys;print(sys.version)"
python3 -I -c "import importlib.metadata as m;print(m.version('jsonschema'))"
```

实测 Python 3.12.14；jsonschema 4.26.0；ZIP 独立解包成功

验证 两个版本命令正常打印，进入 replay 后再做下面步骤

2026-10-05 10:36:45 UTC | 0.142 秒；退出码 0

02 核对上游文件与 schema

目的 验证来源指纹，确认两份 Draft-07 契约可加载

```
python3 -I validate_offline.py --verify-only
```

实测 4 份上游文件指纹匹配；2 份 schema 检查通过

验证 输出 source_files_verified=4、schemas_checked=2、failed=0

2026-10-05 10:36:45 UTC | 0.281 秒；退出码 0

03 让配置正例和反例一起跑

目的 比较类型、枚举、未知字段的不同结果

```
python3 -I validate_offline.py --suite config --output evidence/recheck_config.json
```

实测 9/9 符合预期：3 接受、6 结构拒绝

验证 查看 recheck_config.json：model=42 的错误为 not of type string

2026-10-05 10:36:46 UTC | 0.279 秒；退出码 0

六步实验 从反例到完整报告

04 观察协议形状与语义陷阱

目的 区分请求形状和真实握手、资源与模型资格

```
python3 -I validate_offline.py --suite protocol --output evidence/recheck_protocol.json
```

实测 13/13 符合预期：5 接受、8 结构拒绝

验证 不存在的模型名和 threadId 仍可合规；不代表服务成功

2026-10-05 10:36:46 UTC | 0.330 秒；退出码 0

05 把解析错误单独识别出来

目的 确认坏 JSON/TOML 在结构校验之前就会失败

```
python3 -I validate_offline.py --suite malformed --output evidence/recheck_malformed.json
```

实测 3/3 符合预期，均由解析器报错

验证 查看 recheck_malformed.json：JSONDecodeError 或 TOMLDecodeError

2026-10-05 10:36:46 UTC | 0.213 秒；退出码 0

06 全量回归并保存证据

目的 核对总数，生成可供另一人检查的完整结果

```
python3 -I validate_offline.py --suite all --output evidence/recheck_all.json
```

实测 25/25 符合预期：8 接受、14 结构拒绝、3 解析错误

验证 recheck_all.json 中 passed=25、failed=0；保留完整错误路径

2026-10-05 10:36:46 UTC | 0.279 秒；退出码 0

结果怎么读和知识怎样复用

正例 证明预期结构被接受

25 项样例全部符合预期，其中 8 项被接受。配置空对象、合成 Provider 以及部分协议请求形状可以通过；这是本地校验器的判断，没有被真实运行时采用。

反例 证明错误能被定位

14 项被 schema 拒绝，另有 3 项在 JSON/TOML 解析阶段报错。数字模型名、字符串布尔值、未知枚举、缺失必填项和坏文本可定位到不同错误层。

语义陷阱 证明不能过度解释

协议测试接受了合成模型名和不存在的 threadId，还接受了本包测试的额外 envelope/ClientInfo 字段；配置测试则拒绝未知字段。结果不能推广为全局兼容规则，也不能证明服务、权限或模型可用。

知识查重的实际范围

历史公开知识范围为 394 条索引元数据与 24 篇实际全文阅读的相关公开笔记，其余 370 篇未读正文。该历史范围未发现同主题完整项目；不代表全库无重复或读者已掌握。

已有概念直接复用

沿用“证据状态机”“代码管边界提示词管判断”“单一馈送与 schema 冻结”“沙箱与审批正交”。这次补的是固定版 Codex 契约与反例，不重新讲一遍通用 MCP 或配额工具。

复用前核对同义知识

学习笔记保留公开来源、补充建议与双向关联。复用时先核对目标知识库中的同义条目与已有编辑，再补充本例的固定版本、结果和未测项。

来源 版本与后续验收

登录方式与计费边界

<https://learn.chatgpt.com/docs/auth>

Codex Cloud

<https://learn.chatgpt.com/docs/cloud>

配置基础

<https://learn.chatgpt.com/docs/config-file/config-basic>

高级配置

<https://learn.chatgpt.com/docs/config-file/config-advanced>

App Server 协议

<https://learn.chatgpt.com/docs/app-server>

ChatGPT 登录集成

<https://developers.openai.com/siwc/token-sharing-open-source/codex-app-server>

固定版 ConfigToml schema

<https://github.com/openai/codex/blob/823ea830c0fd418b09ff02d36cad9a1fff66465b/codex-rs/core/config.schema.json>

固定版 ClientRequest schema

<https://github.com/openai/codex/blob/823ea830c0fd418b09ff02d36cad9a1fff66465b/codex-rs/app-server-protocol/schema/json/ClientRequest.json>

上游 Apache 2.0 许可

<https://github.com/openai/codex/blob/823ea830c0fd418b09ff02d36cad9a1fff66465b/LICENSE>

动态文档与固定源码分开

官方产品文档是 2026-10-05 的核对结果，页面可能重定向到 learn.chatgpt.com。schema 和许可固定到明确提交；它们不代表安装清单中 0.159.2 的二进制行为。该清单只读过，二进制没有执行。

后续在线验收需要新证据

若以后获准做真实集成，应分别记录客户端和运行时版本、握手与状态、配置加载结果、账户资格、实际请求与用量归属。不得从本次离线校验推导这些结果。

交付与复验

公开 PDF 直接生成并逐页渲染检查；HTML 仅结构核查。HTML 浏览器及转 PDF 原路线受阻，本次未重试。ZIP 保留许可、schema、样例与运行器；完整历史命令和结果见实验日志。